Q-Compass: Grounding Sequence Mixing in Reinforcement Learning Navigation

Syed Abdur Rehman Ali
Independent Researcher
github.com/Abd0r

March 2026

Abstract

We introduce **Q-Compass**, a drop-in replacement for the standard multi-head attention mechanism that eliminates the Value projection matrix and regrounds sequence mixing in reinforcement learning theory. Standard attention computes routing weights via query-key similarity and mixes *projected* values—four projection matrices in total. Q-Compass instead computes a navigation matrix $Q(s, a) = \text{softmax}(\text{state}(x) \cdot \text{action}(x)^\top / \sqrt{r})$ and gathers *directly* from x, requiring only three projections. The mechanism is formally identical to the $Q(s, a)$ function in RL: *"in state s, how valuable is attending to position a?"* We describe the mechanism, analyse its parameter efficiency, and present **Quatrix** — the architecture built on Q-Compass — along with **Tesseractor**, an earlier proof-of-concept that preceded Quatrix and validated the core idea at small scale before the full multimodal architecture was developed. Both train stably from random initialisation, demonstrating that the Value projection is not necessary for gradient-based language modelling.

1 Introduction

The Transformer [Vaswani et al., 2017] has dominated sequence modelling for nearly a decade. Its core operation—scaled dot-product attention—is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{1}$$

where $Q = xW_Q$, $K = xW_K$, $V = xW_V$ are learned projections and W_O projects the output, totalling *four* parameter matrices per layer. The mechanism is grounded in *similarity*: attend to positions whose keys look like the current query, then mix their projections.

We ask a different question: *what if sequence mixing were grounded in value rather than similarity?*

Reinforcement learning defines the Q-function as $Q(s, a)$—the expected return of taking action a in state s. Applied to sequences: each token position i is a *state* ("where am I in the sequence? what context do I carry?") and each other position j is a *potential action* ("what information is available there?"). The scalar $Q(i, j)$ answers: *how valuable is it to attend to position j from position i?*

This reframing yields Q-Compass, a mechanism that:

- Eliminates W_V, reducing per-layer attention projections from 4 to 3
- Gathers directly from raw hidden states x rather than projected values

- Is formally grounded in RL theory rather than geometric similarity
- Is a drop-in replacement for multi-head attention

We validate the mechanism through **Quatrix**, the architecture built on Q-Compass, and **Tesseractor**, the earlier proof-of-concept that preceded it, and provide theoretical analysis of parameter efficiency and inductive biases.

2 Related Work

Attention variants. Numerous works modify the attention mechanism: linear attention [Katharopoulos et al., 2020], sparse attention [Child et al., 2019], FlashAttention [Dao et al., 2022], and grouped-query attention [Ainslie et al., 2023]. All retain the QKV projection structure. Shen et al. [2021] replace softmax but keep all four projections. To our knowledge, no prior work eliminates W_V as an architectural decision while preserving the standard input/output interface.

RL-grounded architectures. Parisotto et al. [2020] apply RL techniques to improve transformer training stability. Ritter et al. [2021] interpret attention as associative memory. Our contribution differs: we derive the *routing computation itself* from Q-function theory, not merely the training procedure.

Parameter-efficient architectures. LoRA [Hu et al., 2022] and adapters reduce fine-tuning cost via low-rank approximations. Q-Compass achieves parameter reduction *architecturally*—fewer projections by design, with no approximation of any existing projection.

3 Q-Compass

3.1 Mechanism

Given an input $x \in \mathbb{R}^{B \times L \times H}$, Q-Compass computes:

$$\text{state} = xW_s \in \mathbb{R}^{B \times L \times r} \quad (\text{“where am I?”}) \tag{2}$$

$$\text{action} = xW_a \in \mathbb{R}^{B \times L \times r} \quad (\text{“where can I go?”}) \tag{3}$$

$$Q(s, a) = \frac{\text{state} \cdot \text{action}^\top}{\sqrt{r}} \in \mathbb{R}^{B \times L \times L} \tag{4}$$

$$\text{weights} = \text{softmax}(Q(s, a) + M) \tag{5}$$

$$\text{output} = W_o\,(\text{weights} \cdot x) \tag{6}$$

where r is the compass rank (analogous to head dimension), M is the causal mask (lower-triangular for autoregressive models, absent for encoders), and $W_s, W_a \in \mathbb{R}^{H \times r}$, $W_o \in \mathbb{R}^{H \times H}$.

The critical difference from standard attention: *there is no* W_V. The content gathered at each position is x itself—raw and unmodified. All routing intelligence is concentrated in $Q(s, a)$.

3.2 Comparison with Standard Attention

3.3 Intuition: Routing vs. Content Decoupling

Standard attention separates two concerns: *where to look* (controlled by W_Q, W_K) and *what to extract* (controlled by W_V). This gives the model two independent degrees of freedom: it can attend

Mechanism	Projections	Content gathered	RL grounding
Standard Attention	4 (W_Q, W_K, W_V, W_O)	xW_V (projected)	None
Q-Compass	3 (W_s, W_a, W_o)	x (raw)	$Q(s, a)$

Table 1: Structural comparison. Q-Compass eliminates W_V, forcing the model to learn *where* to attend with full precision since it cannot transform content independently of routing.

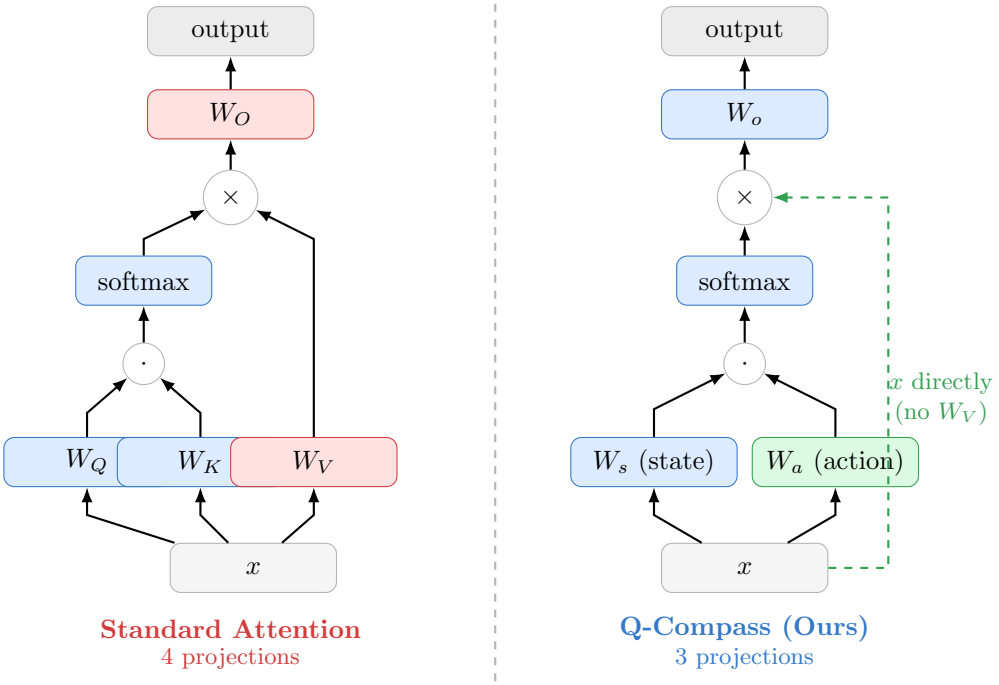

Figure 1: **Standard attention vs. Q-Compass.** Standard attention uses four projections (W_Q, W_K, W_V, W_O) and mixes projected values xW_V. Q-Compass uses three projections (W_s, W_a, W_o) and gathers directly from x (dashed green arrow), eliminating W_V. The navigation matrix $Q(s, a)$ is grounded in RL Q-function theory.

to one position and retrieve a quite different representation via W_V.

Q-Compass collapses these into one: attending to position j delivers exactly the raw representation at j. The routing weights must therefore be precise—there is no W_V to compensate for imprecise attention patterns. We hypothesize this encourages the model to learn more structured, interpretable navigation patterns in the weight matrices W_s and W_a.

3.4 Bidirectional Variant (Q-Compass-Bi)

For non-autoregressive contexts (e.g., image patch encoding), Q-Compass operates without the causal mask, yielding full bidirectional navigation:

$$Q_{\text{bi}}(s, a) = \text{softmax}\left(\frac{\text{state}(x) \cdot \text{action}(x)^\top}{\sqrt{r}}\right) \tag{7}$$

The identical computation with a different mask allows a single mechanism to serve both autoregressive language modelling and vision encoding.

4 Architecture

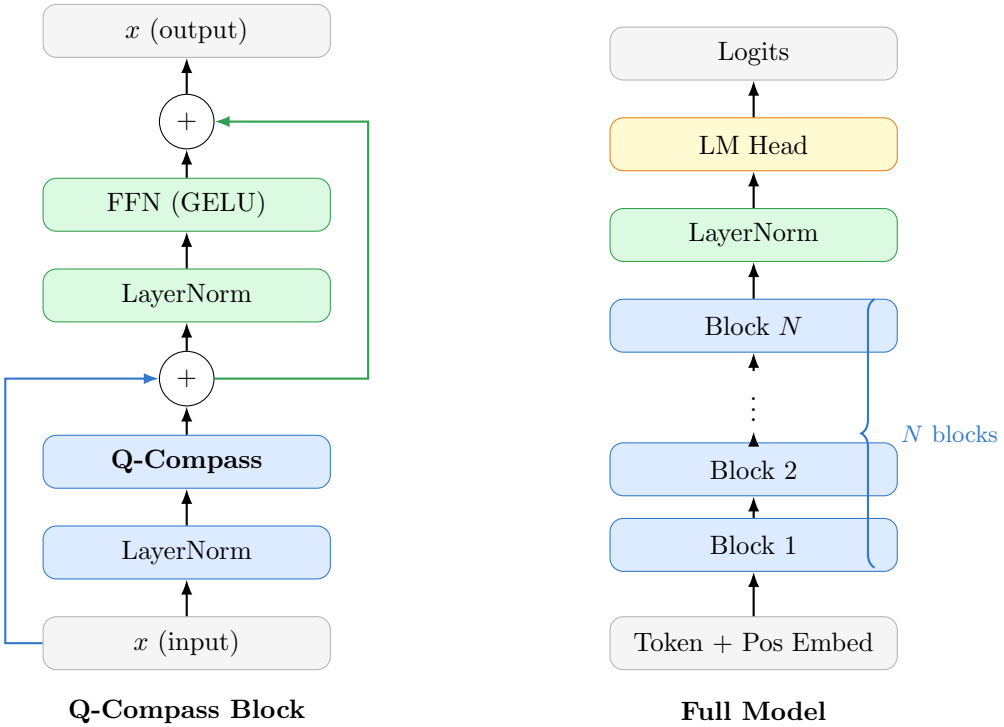

Figure 2: **Left**: Single Q-Compass block with pre-norm residual connections. Blue skip bypasses Q-Compass; green skip bypasses FFN. **Right**: Full model stack, shared by Tesseractor (PoC) and Quatrix (full architecture).

4.1 Tesseractor: Early Proof of Concept

Tesseractor is the predecessor to Quatrix — a text-only decoder-only language model built to validate that Q-Compass supports stable training from random initialisation before the full Quatrix architecture was developed.

Hyperparameter	Value
Parameters	50.2M
Hidden size	512
Layers	10
Compass rank r	64
FFN ratio	4×
Context length	256 tokens
Vocabulary	50,257 (GPT-2 BPE)

Table 2: Tesseractor configuration.

Block structure. Each block applies pre-norm residual updates:

$$x \leftarrow x + \text{QCompass}(\text{LN}(x)) \tag{8}$$

$$x \leftarrow x + \text{FFN}(\text{LN}(x)) \tag{9}$$

with GELU activation in the FFN.

Q-value language modelling head. Beyond the sequence mixer, we extend the RL framing to next-token prediction. Let h_i be the hidden state at position i and e_v the embedding of vocabulary token v. Tesseractor's LM head predicts:

$$\text{logits}(i, v) = \frac{(W_s^{\text{lm}}\, h_i) \cdot (W_a^{\text{lm}}\, e_v)^\top}{\sqrt{r}} \tag{10}$$

This treats next-token selection as a Q-value estimation: *"in the current context state h_i, how valuable is emitting token v as the next action?"* The model is therefore fully RL-grounded end-to-end: Q-values govern both attention routing and output selection.

Training. Tesseractor was trained from scratch on approximately 1M text samples (web, code, math, and instruction data) for 3 epochs, using AdamW with cosine learning rate decay. Training loss converged smoothly, confirming gradient flow is unimpeded by the absence of W_V.

4.2 Quatrix: The Full Architecture

Quatrix is the primary architecture built on Q-Compass, refining and extending the lessons from Tesseractor into a full multimodal system.

Hyperparameter	Value
Parameters (LM)	44M
Parameters (Vision)	5.5M
Parameters (Proj.)	0.4M
Total	∼50M
Hidden size	512
LM layers	7
Vision layers	3 (Q-Compass-Bi)
Compass rank r	64
Context length	5,120 tokens

Table 3: Quatrix configuration.

Key differences from Tesseractor.

1. **LM head**: Replaced by tied embeddings (standard linear projection), isolating the Q-Compass contribution to sequence mixing.
2. **Context**: Extended from 256 to 5,120 tokens.
3. **Vision**: A lightweight Q-Compass-Bi encoder processes image patches and prepends them to the token sequence.

Vision encoder. Images are patchified into 16×16 patches (196 patches per 224×224 image), processed by 3 bidirectional Q-Compass blocks ($H = 384$, $r = 32$), then projected to the LM hidden dimension. No external vision model (CLIP or similar) is used. The same mathematical operation—with or without the causal mask—handles all modalities uniformly.

Unified mechanism. This uniformity is a key property: a single implementation of $Q(s, a)$ serves causal text generation and bidirectional image encoding. There is no separate attention kernel for vision vs. language, simplifying the architecture and reducing the total number of novel components.

5 Analysis

5.1 Parameter Efficiency

Per-layer attention parameter counts:

$$\text{Standard attention:} \quad 4H^2 \text{ parameters} \quad (W_Q, W_K, W_V, W_O \in \mathbb{R}^{H \times H}) \tag{11}$$

$$\text{Q-Compass:} \quad 2Hr + H^2 \text{ parameters} \quad (W_s, W_a \in \mathbb{R}^{H \times r},\ W_o \in \mathbb{R}^{H \times H}) \tag{12}$$

For $H = 512$, $r = 64$, standard attention requires 1,048,576 parameters per layer; Q-Compass requires 327,680—a **69% reduction** in attention-block parameters. FFN layers are unchanged. The rank r is a tunable hyperparameter that directly controls this tradeoff.

5.2 Drop-in Compatibility

Q-Compass preserves the input/output signature of standard attention: $\mathbb{R}^{B \times L \times H} \to \mathbb{R}^{B \times L \times H}$. It can replace multi-head attention in any existing transformer with no changes to surrounding layers, norms, or optimisers.

5.3 Expressivity

Standard attention with W_V can represent any linear mixing of x-projections. Q-Compass restricts the gathered content to x itself, but the softmax routing weights remain unconstrained. Any weighted average of input vectors x is still reachable; what is lost is the ability to *simultaneously* route to position j and retrieve a different projection of that position.

Formally, Q-Compass computes:

$$\text{output}_i = W_o \sum_j \alpha_{ij}\, x_j, \quad \alpha_{ij} = \text{softmax}\left(\frac{(W_s x_i) \cdot (W_a x_j)^\top}{\sqrt{r}} \right) \tag{13}$$

This is a *content-invariant* mixer: the routing α depends on both i and j, but the content x_j is unmodified before mixing. W_o then projects the mixed output, providing one linear transformation after aggregation.

5.4 Inductive Bias

By forcing the model to attend to and gather raw x, Q-Compass creates a tighter coupling between routing and representation. The model cannot learn to attend broadly and rely on W_V to "filter" irrelevant information post-hoc. Instead, W_s and W_a must encode the full routing policy, potentially encouraging more disentangled state and action representations.

Whether this inductive bias is beneficial across tasks and scales is an empirical question we leave to future work.

6 Discussion

Why might W_V be removable? W_V in standard attention serves to project the gathered content before mixing. However, W_o already performs a linear projection *after* mixing. The composition $W_o \cdot W_V$ is a single linear transformation that could, in principle, be absorbed into W_o if the routing weights are expressive enough. Q-Compass makes this explicit: skip W_V, let W_o handle the post-mix transformation.

Connection to linear attention. Linear attention [Katharopoulos et al., 2020] approximates $\text{softmax}(QK^\top)V$ with a kernel to reduce quadratic cost. Q-Compass instead *replaces* V with x and retains exact softmax. The two directions are orthogonal and could be combined.

Scaling. Q-Compass has not been evaluated at large scale ($\geq$1B parameters). At small compass rank r, the state and action projections are a bottleneck; setting $r = H$ recovers the full-rank regime. The regime $r \ll H$ offers significant parameter savings but may limit expressivity. Scaling laws for this tradeoff are unknown.

Multi-head extension. Standard attention uses multiple heads to capture diverse routing patterns. Q-Compass can be extended analogously by partitioning H into n_h groups of dimension H/n_h, each with its own $W_s^{(k)}, W_a^{(k)}$. We leave multi-head Q-Compass to future work.

7 Conclusion

We presented **Q-Compass**, a drop-in replacement for transformer attention that eliminates the Value projection matrix and regrounds sequence mixing in the RL $Q(s, a)$ framework. The mechanism reduces attention-block parameter count by 69% (at $H = 512$, $r = 64$) while preserving the module's input/output interface. We demonstrated stable training through Tesseractor (the early PoC) and Quatrix (the full architecture), confirming that language modelling does not require a Value projection.

The central claim of this paper is theoretical: *routing (where to attend) and extraction (what to gather) can be collapsed into a single RL-grounded operation without breaking gradient flow or expressivity in practice.* We hope this framing opens new directions for interpreting and improving sequence mixing mechanisms.

Future work. **Berry-Q0** is the first model trained on the Quatrix architecture, currently undergoing R1-style GRPO reasoning training [Guo et al., 2025] on mathematics. Empirical results—including reasoning benchmark evaluations and an analysis of whether RL-grounded sequence mixing affects the emergence of reasoning—will be reported in a subsequent paper once training is complete.

Acknowledgments

This work was conducted independently, without institutional affiliation or external funding.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zelaski, Sumit Sanghai, and Yunheng Xu. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 2022.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. *International Conference on Machine Learning*, 2020.

Emilio Parisotto, H Francis Song, Jack W Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, et al. Stabilizing transformers for reinforcement learning. *International Conference on Machine Learning*, 2020.

Samuel Ritter, Jane X Wang, Zeb Kurth-Nelson, and Matthew Botvinick. Rapid task-solving in novel environments. *International Conference on Learning Representations*, 2021.

Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.